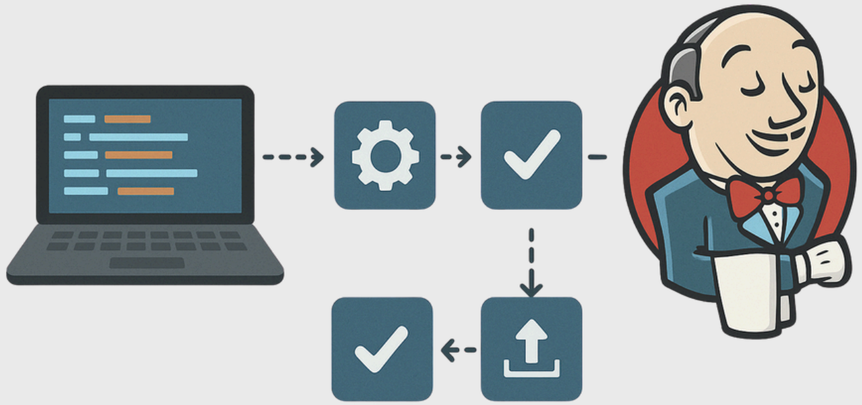
Université de Lomé

Ecole Polytechnique de Lomé

utbm
université de technologie Belfort-Montbéliard



Mise en place de l'intégration continue avec Jenkins

Auteurs : KOLANI Julien Yendoupabe
TELOU Abire Jennefer

Superviseur : Amir HAJJAM EL HASSANI

Plan de la présentation

1 Introduction à l’intégration continue

2 Principes fondamentaux de l’intégration continue (CI)

3 Les avantages de l’Intégration Continue

4 Outils d'Intégration Continue

5 Présentation de Jenkins

6 Historique et origine de Jenkins

7 Installation et configuration de Jenkins

8 Création d’un pipeline d’intégration continue

9 Automatisation du build, des tests et du déploiement

10 Plugins et extensions pour Jenkins

11 Mise en œuvre de l’Intégration Continue

12 Démonstration

13 Bonnes pratiques

14 Ouverture : vers la livraison continue (CD)

15 Conclusion

1 Introduction à l’intégration continue

Dans le cadre du développement logiciel, la rapidité, la qualité et la capacité d’adaptation sont devenues des exigences clés pour rester compétitif, produire du bon logiciel respectueux des délais et attirer de nouveaux clients. Pour répondre à ces enjeux, de nombreuses équipes adoptent les méthodes agiles. Cette méthode permet une grande réactivité quant aux demandes du client et augmente par conséquent sa satisfaction.

L’une des pratiques les plus populaires issues de la méthode agile est l’intégration continue (IC).

1 Introduction à l’intégration continue

L'intégration continue est un ensemble de pratiques qui permettent de s'assurer que les nouvelles fonctionnalités qu'on intègre à une application en cours de développement ne rendent pas non fonctionne le code déjà existant. C'est l'un des fondamentaux du développement logiciel moderne, elle change radicalement la manière dont les équipes pensent le processus de développement. Elle permet d'induire toute une série d'améliorations et de transformations, depuis la construction automatisée de l'application jusqu'à la livraison continue en production.

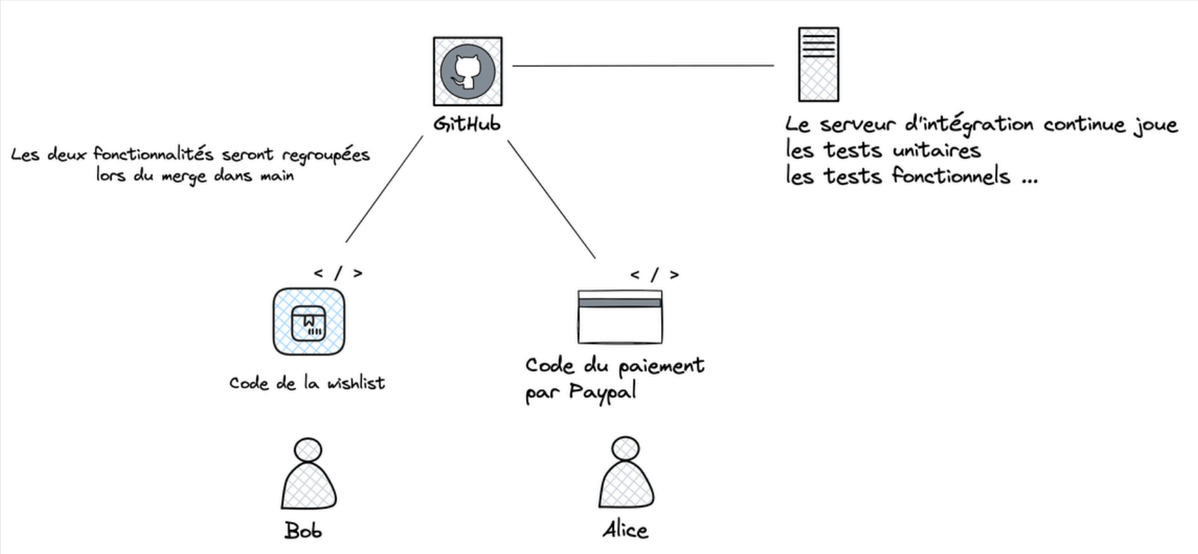
Illustration

Orlog et son équipe de développeurs travaillent sur de nouvelles fonctionnalités pour le site web de l’université de Lomé. Chacun des développeurs a travaillé sur des branches séparées durant la semaine écoulée. Maintenant, ils sont tous confiants dans leur code et prêts à fusionner leurs modifications.
Ils lancent le processus de fusion… et voient leur console se remplir de conflits !
Orlog pousse un soupir : il avait des projets pour ce week-end.

2 Principes fondamentaux de l’intégration continue (CI)

L’intégration continue vise à accélérer le développement des logiciels en automatisant l’intégration du code afin d’éviter les erreurs liées aux fusions tardives des différentes parties du code, développées indépendamment les unes des autres. [1]

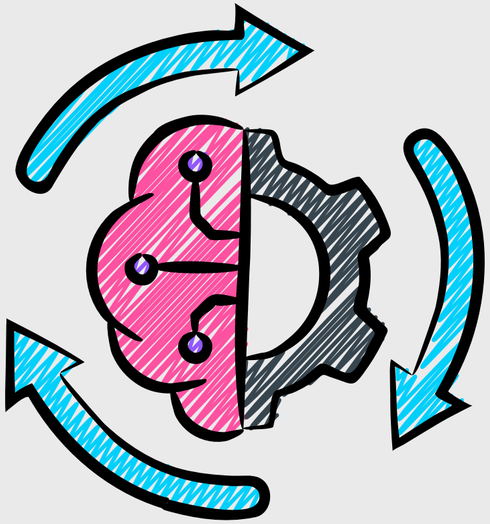
L’objectif de l’intégration continue est de vérifier qu’à chaque modification de code, le build de l’application reste possible et qu’il n’y a pas de régressions provoquées. Pour y parvenir, plusieurs règles sont définies.[1]



2 Principes fondamentaux de l’intégration continue (CI)

L'Intégration Continue s'appuie principalement sur :

- **Hébergement centralisé du code**
- **Commits fréquents et incrémentaux**
- **Automatisation des builds**
- **Exécution automatique des tests**



3 Les avantages de l'Intégration Continue

Optimisation des cycles de développement

- Gain de temps et efficacité accrue
- Livraison plus fréquente et plus fiable

Amélioration de la qualité du code

- Tests automatisés à chaque modification
- Suivi constant de l'état du code

Amélioration indirecte du référencement web

- Mises à jour fréquentes et qualitatives

Détection rapide des erreurs

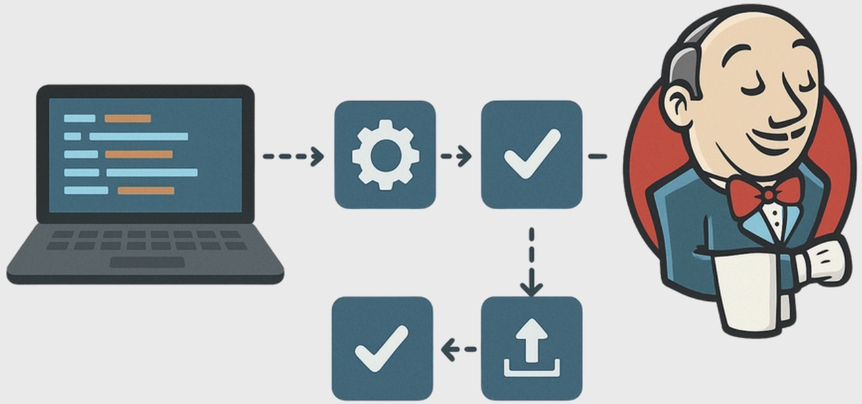
- Réduction du temps de détection des bugs

4 Outils d'Intégration Continue

Caractéristique	Jenkins	Travis CI	GitLab CI/CD
Licence	**Open Source**	Freemium *(gratuite pour open source)*	Open Source
Langage principal	**Java**	Ruby	Ruby
Pipeline as Code	✅ **Oui**	✅ Oui	✅ Oui
Parallélisation	✅ **Oui**	✅ Oui	✅ Oui
Support des conteneurs	✅ **Oui**	✅ Oui	✅ Oui
Plugins et extensions	**Très nombreux**	Quelques-uns	Intégré à GitLab
Intégration Git	**Via plugins**	Native avec GitHub	Native avec GitLab
Installation séparée requise	**Oui**	Non (SaaS)	Non *(intégré à GitLab)*
URL officielle	**jenkins.io**	travis-ci.com	gitlab.com

5 Présentation de Jenkins

Jenkins est un outil d'intégration continue qui permet d’effectuer des tests et un déploiement fluides et continus du code nouvellement créé. Ceci permet de réduire la charge de travail des testeurs, accélère l'intégration et réduit le gaspillage de ressources.



Jenkins est l'un des meilleurs, car il est gratuit, open source et s’intègre à majeure partie des autres outils DevOps.

Plus d'un millier de plugins permettent d'étendre les fonctionnalités de Jenkins et de le rendre plus personnalisé. Tous ces plugins et extensions sont développés en Java. Ainsi, Jenkins peut également être installé sur tout système d'exploitation exécutant Java.

6 Historique et origine de Jenkins

L'histoire de Jenkins commence en 2004 avec la création d' **Hudson** par **Kohsuke Kawaguchi**. En 2009 Oracle rachète Sun Microsystems devenant ainsi le propriétaire du projet. En Fin d'année 2010 des tensions apparaissent entre la communauté et Oracle, conduisant en janvier 2011 à une copie du projet qui donne naissance à Jenkins. Dès février 2011, Jenkins publie sa première version officielle. [2]

Le projet connaît une croissance rapide, atteignant 50 000 installations actives en 2013. En 2016 Jenkins est transféré à la Continuous Delivery Foundation pour assurer sa gouvernance. l'année 2017 marque le lancement de Jenkins 2.0 avec des améliorations significatives. En 2018, Jenkins dépasse les 150.000 installations et célèbre ses 10 ans en 2020. En 2024 Jenkins intègre des fonctionnalités d'IA et de machine learning pour améliorer les pipelines CI/CD.

Kohsuke Kawaguchi.

- **Pourquoi Jenkins a autant marqué ?**

C'est l'un des premiers outils open source, extensibles et universels pour la CI/CD.
Avant Jenkins, il fallait tout bricoler à la main avec des scripts d'automatisation.

7 Installation et configuration de Jenkins

Prérequis

- Java (JDK 11 ou supérieur)
- Accès admin sur la machine
- Docker (pour une installation via docker)

🌐 **Accès Web**

- Naviguer vers http://localhost:8080
- Entrer le mot de passe initial (généré au lancement)
- Installer les plugins recommandés

```
2f064d3663814887964b682940572567

This may also be found at: /var/jenkins_home/secrets/initialAdminPassword
```

Sous Linux (Debian/Ubuntu)

- wget -q -O - https://pkg.jenkins.io/debian/jenkins.io.key | sudo apt-key add -
- sudo sh -c 'echo deb https://pkg.jenkins.io/debian binary/ > /etc/apt/sources.list.d/jenkins.list'
- sudo apt update
- sudo apt install jenkins
- sudo systemctl start jenkins

👤 Création de l'utilisateur admin → 🧩 Installation de plugins essentiels → 📁 Configuration d'un job

8 Création d'un pipeline d'intégration continue

Le pipeline Jenkins est défini dans un fichier texte nommé Jenkinsfile. Celui-ci respecte une syntaxe stricte appelée Groovy. On y définit les différentes étapes de l'intégration continue que nous souhaitons implémenter (build, test, déploiement…).

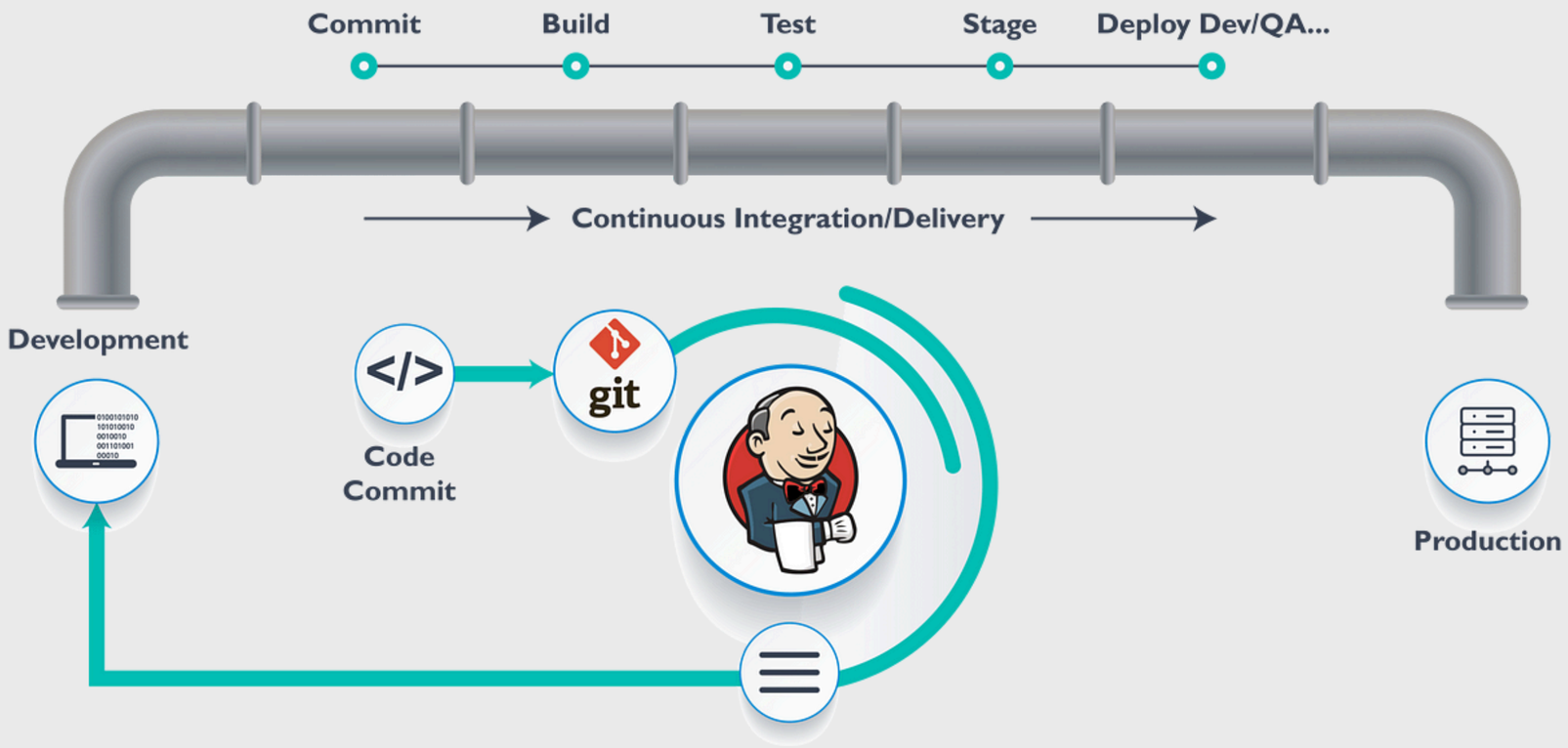
Élément constitutif du pipeline

- **Node : agent Jenkins où s'exécutent les étapes**
- **Stage : phase logique (ex. : Build / Test / Deploy)**
- **Step : action concrète (ex. : sh 'npm test')**

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps { sh 'make' }
    }
    stage('Test') {
      steps {
        sh 'make check'
        junit 'reports/**/*.xml'
      }
    }
  }
}
```

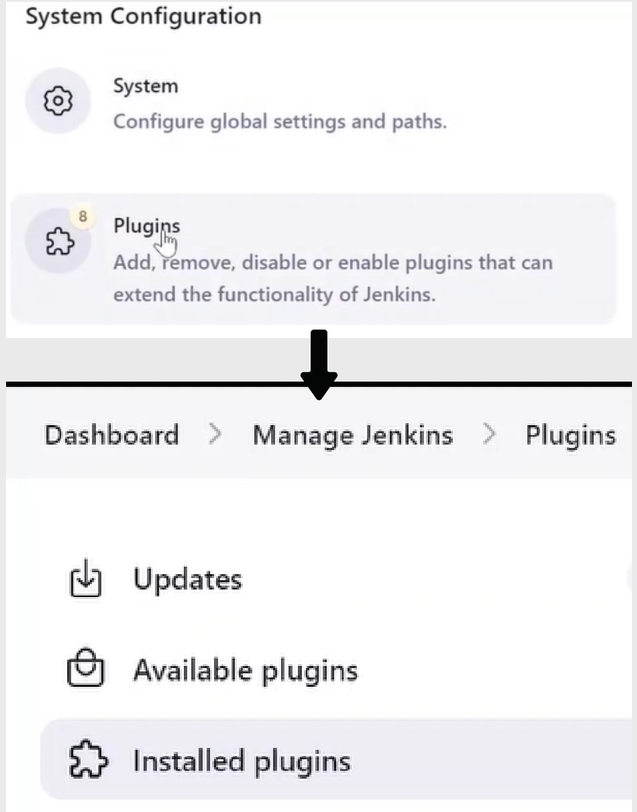
🧩 Pipeline déclaratif en groovy

9 Automatisation du build, des tests et du déploiement



Élément constitutif du pipeline [3]

10 Plugins et extensions pour Jenkins



Les plugins Jenkins font parties intégrantes de l'environnement Jenkins, ce sont des petits programmes indépendants qui permettent d'améliorer et d'attendre les fonctionnalités de Jenkins.

Plugins Jenkin populaires

- Plugin Git
- Plugin Docker
- Plugin de pipeline
- Plugin de métriques
- Plugin Blue Ocean

11 Mise en œuvre de l'Intégration Continue

L'intégration continue ne se limite pas à un concept : c'est une pratique concrète qui repose sur des outils et une organisation rigoureuse du processus de développement. Avant de passer à la démonstration, posons les bases.

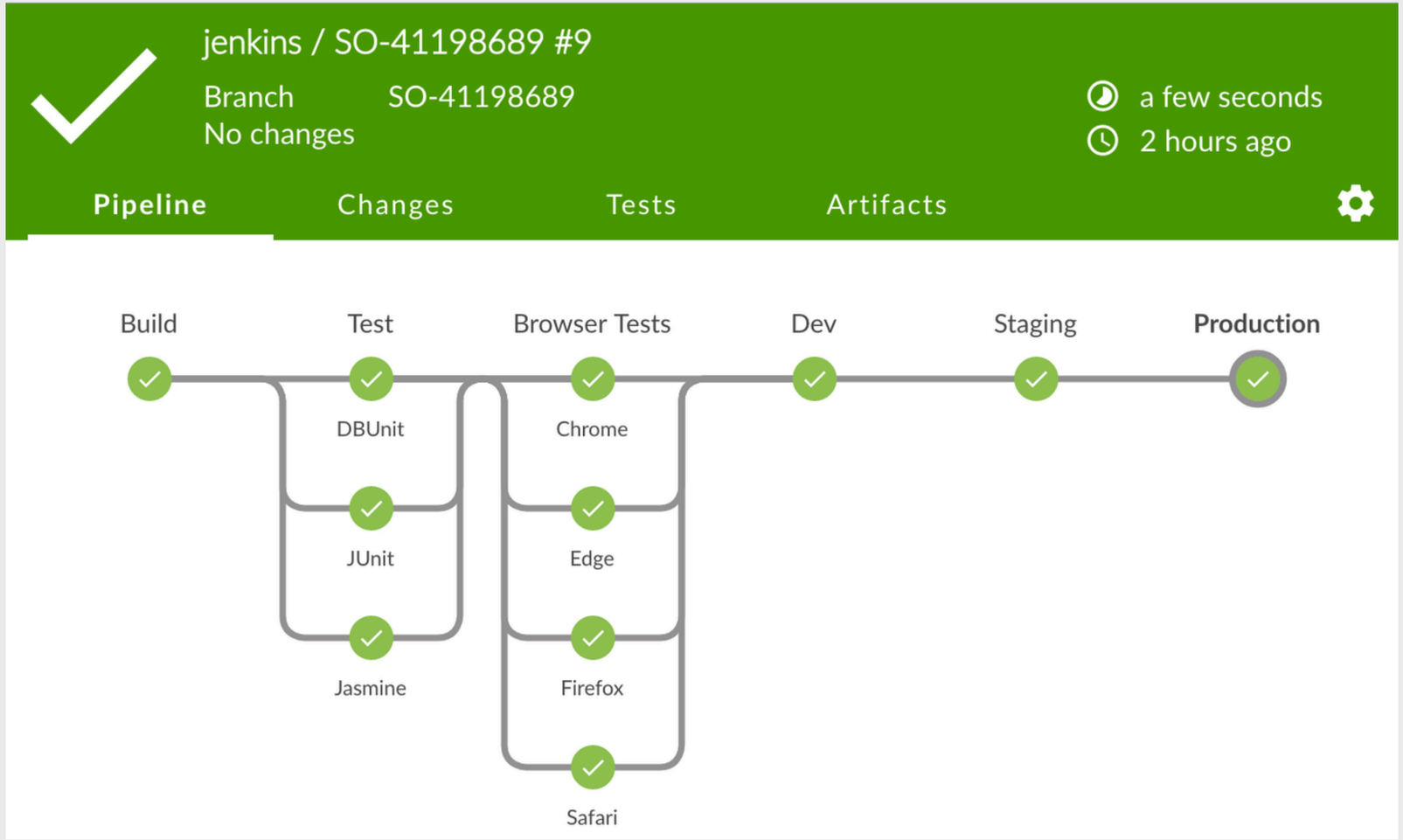
Les prérequis

- Un dépôt de code versionné
- Des tests unitaires
- Un serveur CI
- Un environnement stable de build

Étapes de mise en place

- Connecter le dépôt de code au serveur CI
- Structurer les tests
- Configurer les jobs CI
- Gérer les échecs de tests

12 Démonstration



13 Bonnes pratiques

1. Automatiser la définition des jobs
2. Utiliser des dossiers d'organisation
3. Utiliser des Pipelines
4. Gérer vos jobs
5. Afficher les résultats des builds
6. Utiliser des agents pour les builds
7. Alerter les bonnes personnes en cas d'échec
8. Choisir des noms simples pour les projets
9. Utiliser la "Fingerprint" pour vos dépendances
11. Gérer le contrôleur
12. Éviter la surcharge de planification
13. Éviter les collisions de ressources [4]

14 Ouverture : vers la livraison continue (CD)

Au-delà de l’intégration continue, Jenkins permet aussi d’automatiser la livraison des produits logiciels, à savoir dans les environnements de test, préprod et de production

Ceci permet aussi de restaurer, en cas d’échec ou d’erreurs, la récupération rapide d’une ancienne version du logiciel. [5]

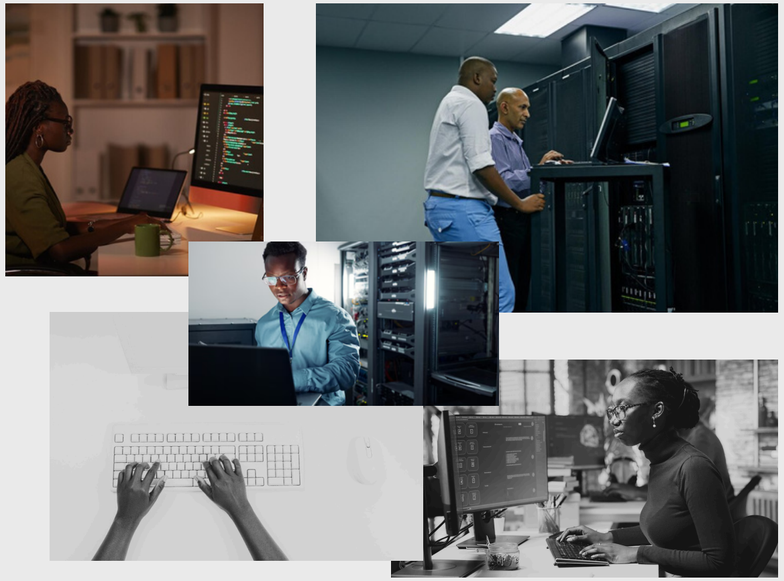
15 Conclusion

L’intégration continue s’impose comme une pratique essentielle du développement logiciel. Grâce à la puissance de Jenkins, sa modularité et sa large communauté il s’affirme comme l’un des outils les plus complets et modulaires pour automatiser les étapes critiques du cycle de vie du logiciel.

L’installation de Jenkins, la configuration des jobs, l’écriture des pipelines et l’automatisation des tests et déploiements permettent d’améliorer significativement la qualité du code et la réactivité des équipes de développement. L’usage de pipelines déclaratifs et de plugins spécialisés apporte structure, fiabilité et gain de temps au quotidien.

Au-delà de l’outil c’est l’organisation des équipes la rigueur des tests et la culture DevOps qui donnent tout son sens à cette démarche. Jenkins n’est pas une fin en soi mais c’est un levier pour tendre vers une livraison continue plus fiable, et plus réactive aux besoins des utilisateurs.

Merci de votre attention



Annexe et bibliographie

Bibliographie

[1] S. Médard, “Serveurs d'intégration continue Jenkins et d'analyse de code Sonar couplés à la forge logiciel SourceSup,” in Journées Réseaux de l’Enseignement et de la Recherche (JRES), Montpellier, France, 10–13 déc. 2013.
[2] Jenkins-le-guide-complet community, “De Hudson à Jenkins — Un rapide historique,” Jenkins-le-guide-complet.github.io, section 1.4. [En ligne]. Disponible : Les présentations sont des outils de communication pouvant être utilisés comme des démonstrations, des conférences, des discours, des rapports et bien plus encore. Elles sont pour la plupart présentées en public. [Consulté le 1 juin 2025].
[3] Jenkins community, “Pipeline Best Practices,” Jenkins.io. [En ligne]. Disponible : Les présentations sont des outils de communication pouvant être utilisés comme des démonstrations, des conférences, des discours, des rapports et bien plus encore. Elles sont pour la plupart présentées en public. [Consulté le 1 juin 2025].
[4] Simplilearn, “What is Jenkins?”, Simplilearn.com. [En ligne]. Disponible : Les présentations sont des outils de communication pouvant être utilisés comme des démonstrations, des conférences, des discours, des rapports et bien plus encore. Elles sont pour la plupart présentées en public. [Consulté le 2 juin 2025].
[5] LeTecode, “Comment devenir un développeur web full stack en 2022,” LeTecode.com. [En ligne]. Disponible : Les présentations sont des outils de communication pouvant être utilisés comme des démonstrations, des conférences, des discours, des rapports et bien plus encore. Elles sont pour la plupart présentées en public. [Consulté le 2 juin 2025].
[6] LeTecode, “Infographie CI/CD,” LeTecode.com. [En ligne]. Disponible : Les présentations sont des outils de communication pouvant être utilisés comme des démonstrations, des conférences, des discours, des rapports et bien plus encore. Elles sont pour la plupart présentées en public. [Consulté le 3 juin 2025].